



TP PHP : Les Tableaux Multidimensionnels

BTS SIO – Bloc 1 : Support et mise à disposition de services informatiques

1. Contexte du TP

Dans le cadre du développement d'un module d'inventaire pour une société de services numériques (ESN), vous devez manipuler des données stockées sous forme de **tableaux de tableaux** (communément appelés "Recordsets" lorsqu'ils proviennent d'une base de données).

L'objectif est de mettre en pratique l'itération (foreach), l'accès aux clés d'un tableau associatif et le retour de fonctions typées.

2. Structure de données

Le tableau `$inventaire` est un tableau indexé contenant des tableaux associatifs représentant des périphériques informatiques. Chaque produit possède les clés : `id`, `designation`, `prix_unitaire` et `quantite`.

```
$inventaire = [
    ["id" => 1, "designation" => "Ecran 24 pouces", "prix_unitaire" => 150, "quantite" => 12],
    ["id" => 2, "designation" => "Souris sans fil", "prix_unitaire" => 25, "quantite" => 45],
    ["id" => 3, "designation" => "Clavier mécanique", "prix_unitaire" => 80, "quantite" => 8],
    ["id" => 4, "designation" => "Casque audio", "prix_unitaire" => 55, "quantite" => 15],
    ["id" => 5, "designation" => "Serveur NAS", "prix_unitaire" => 450, "quantite" => 2]
];
```

3. Missions (Travail à réaliser)

Mission 1 : Filtrage par budget

Fonction : `getProduitsAbordables(array $table, float $prixMax): array`

- **Description** : Parcourt l'inventaire et extrait uniquement les noms des produits dont le prix est inférieur ou égal au seuil `$prixMax`.
- **Contrainte** : La fonction doit retourner un tableau simple (indexé) de chaînes de caractères.

Mission 2 : Valorisation comptable

Fonction : `calculerValeurTotale(array $table): float`

- **Description** : Calcule la valeur totale du stock disponible.

Mission 3 : Recherche unitaire

Fonction : `getInfosProduit(array $table, int $idCherche): ?array`

- **Description** : Cherche si un produit possédant l'identifiant `$idCherche` existe dans le tableau.
- **Contrainte** : Si le produit est trouvé, retourner son tableau associatif complet. Sinon, retourner null.

Mission 4 : Analyse de stock

Fonction : `getProduitMoinsCher(array $table): ?array`

- **Objectif** : Identifier le produit qui possède le prix unitaire le plus bas dans tout l'inventaire.
- **Contrainte** : La fonction doit retourner le tableau associatif complet du produit trouvé. Si le tableau est vide, elle retourne null.

Mission 5 : Mise à jour globale

Fonction : `appliquerRemise(array $table, float $pourcentage): array`

- **Objectif** : Simuler une période de soldes. La fonction doit retourner un **nouveau tableau** identique à l'original, mais où chaque `prix_unitaire` a été réduit du pourcentage donné en paramètre.
- **Contrainte** : Ne modifiez pas le tableau original, retournez une copie modifiée.

Structure du projet

tp_inventaire/

|— data/

| |— inventaire.php # Source de données : le tableau

|— lib/

| |— fonctions.php # Logique métier

|— tests/

|— test_get_abordables.php

|— test_calcul_valeur.php

|— test_recherche_id.php

|— test_moins_cher.php

|— test_application_remise.php



Guide de nommage : Les bonnes pratiques en PHP

Élément	Convention	Exemple	Règle d'or
Variables	camelCase	<code>\$inventaireStock,</code> <code>\$prixUnitaire</code>	Toujours en anglais ou français constant, mais jamais de majuscule au début.
Fonctions	camelCase	<code>calculerValeur(),</code> <code>getInfos()</code>	Commencer par un verbe d'action . Le nom doit décrire ce que fait la fonction.
Fichiers	snake_case	<code>fonctions.php,</code> <code>gestion_stock.php</code>	Minuscules uniquement. Utiliser le tiret bas <code>_</code> pour séparer les mots.
Constantes	UPPER_CASE	<code>TAUX_TVA,</code> <code>VERSION_APP</code>	Toujours en majuscules avec des tirets bas.
Clés de tableaux	snake_case	<code>\$p['prix_unitaire'],</code> <code>\$p['id_produit']</code>	Souvent calqué sur les noms des colonnes de la base de données (minuscules).

Partie 2

Pour rendre le TP plus réaliste (proche d'une structure de base de données relationnelle), nous allons introduire un tableau **\$categories**.

L'étudiant devra maintenant effectuer des **jointures logiques** et des **regroupements**, ce qui est au cœur du métier de développeur.

1. Mise à jour des données (data/inventaire.php)

```
// Les catégories
$categories = [
    ["id" => 10, "nom" => "Périphériques"],
    ["id" => 20, "nom" => "Stockage"],
    ["id" => 30, "nom" => "Composants"]
];

// L'inventaire mis à jour avec la clé étrangère 'id_categorie'
$inventaire = [
    ["id" => 1, "designation" => "Écran 24 pouces", "prix_unitaire" => 150, "quantite" => 12, "id_categorie" => 10],
    ["id" => 2, "designation" => "Souris sans fil", "prix_unitaire" => 25, "quantite" => 45, "id_categorie" => 10],
    ["id" => 3, "designation" => "Clavier mécanique", "prix_unitaire" => 80, "quantite" => 8, "id_categorie" => 10],
    ["id" => 4, "designation" => "Casque audio", "prix_unitaire" => 55, "quantite" => 15, "id_categorie" => 10],
    ["id" => 5, "designation" => "Serveur NAS", "prix_unitaire" => 450, "quantite" => 2, "id_categorie" => 20],
];
```

2. Les nouvelles missions

Ces missions demandent de manipuler deux tableaux simultanément.

Mission 6 : Recherche du nom de catégorie

Fonction : getNomCategorie(array \$categories, int \$idCat): string

- **Objectif** : À partir d'un ID de catégorie, retrouver son nom libellé.
- **Retour** : Le nom de la catégorie (ex: "Stockage") ou "Inconnue".

Mission 7 : Inventaire détaillé

Fonction : `getInventaireDetaille(array $inventaire, array $categories): array`

Objectif : Générer un nouveau tableau où chaque produit contient une nouvelle clé `nom_categorie` au lieu de l'ID.

Complexité : Utiliser la fonction de la Mission 6 à l'intérieur de la boucle.

Format de sortie : Un tableau indexé de tableaux associatifs (**array 2D**).

Structure d'un élément : Chaque produit doit conserver ses clés d'origine (`id`, `designation`, `prix_unitaire`, `quantite`, `id_categorie`) et posséder une **6ème clé** nommée `nom_categorie`.

Mission 8 : Filtrage par catégorie

Fonction : `getProduitsParCategorie(array $inventaire, int $idCat): array`

- **Objectif** : Retourner tous les produits appartenant à une catégorie précise.
- **Retour** : Un sous-ensemble du tableau `$inventaire`.

Mission 9 : Statistiques de stock

Fonction : `getNombreProduitsParCategorie(array $inventaire): array`

- **Objectif** : Compter combien de références de produits il y a pour chaque ID de catégorie.
- **Retour** : Un tableau associatif sous la forme :

`[id_categorie => nombre]`

Ex: `[10 => 4, 20 => 2]`.